C- Deliverable Document

By Stack Architects

1. Introduction

The Library Management System (LMS) is a web-based application developed to digitize and streamline the operations of a traditional library. The system is designed to support three primary user roles: students, librarians, and administrators.

During **Sprint 1**, the foundational module of the system, namely **Account Management**, was successfully implemented. This included student registration, admin approval/rejection of accounts, role-based login, and account deletion requests.

Building upon this foundation, **Sprint 2** focused on implementing the **core operational features** of the system. These include book catalogue management, book search functionality, borrowing workflow, and due date tracking. Sprint 2 significantly enhanced the system by enabling real-world library operations such as managing inventory and processing borrowing requests.

2. Project Vision

The vision of the Library Management System is to provide a **centralized, efficient, and user-friendly platform** for managing all library-related activities.

The system aims to:

- Replace manual processes with automated workflows
- Improve accessibility of library resources
- Ensure controlled and secure borrowing of books
- Provide transparency in library operations

During Sprint 2, the vision was further refined to emphasize:

- Accurate tracking of individual book copies
- Controlled borrowing through approval workflows
- Efficient search and discovery of books

3. Intended Use of the System

Stakeholders

- Students
- Librarians
- Administrators

Stakeholder Needs

Students:

- Search and view books
- Request books for borrowing
- Track borrowed books and due dates

Librarians:

- Manage book inventory
- Approve or reject borrowing requests
- Track issued books

Administrators:

- Manage user accounts
- Monitor system activity

Updates from Sprint 2

During Sprint 2, stakeholder needs were refined to include:

- Students require visibility of **due dates and borrowing history**
- Librarians require control over **individual book copies and availability**
- The system must enforce **borrowing limits** to ensure fair usage

4. Features and Functionality

Features Completed in Sprint 1

- Student registration (pending admin approval)
- Admin approval/rejection of accounts
- Role-based login system
- Account deletion request

Features Implemented in Sprint 2

- Book catalogue management (add, update, delete books)
- Book search functionality
- Book detail viewing and availability tracking
- Borrow request system
- Borrow approval and rejection by librarian
- Due date assignment for issued books
- Borrow history tracking
- Borrow limit enforcement

Improvements from Sprint 2

Sprint 2 significantly enhanced system capability by:

- Introducing **real-time book inventory management**
- Enabling a **complete borrowing workflow**
- Providing **structured control over resource allocation**
- Allowing users to **track borrowing and due dates**

These improvements transformed the system from a basic account platform into a **fully functional library management solution**.

5. User Stories

Sprint 2 User Stories

- As a librarian, I want to add books so that they become available in the system.
- As a librarian, I want to update book details so that information remains accurate.
- As a librarian, I want to delete books so that outdated records are removed.
- As a student, I want to search books so that I can find relevant titles easily.
- As a student, I want to request a book so that I can borrow it.
- As a librarian, I want to approve or reject borrowing requests so that book issuing is controlled.
- As a system, I want to assign due dates so that borrowing is time-limited.
- As a student, I want to view my borrowed books so that I can track due dates.

Sprint 1 Leftover Stories Completed in Sprint 2

- Integration of role-based access with new modules

- UI consistency improvements across dashboards

6. Non-Functional Requirements (NFR)

The system satisfies the following non-functional requirements:

Performance

- The system shall respond to user requests within acceptable time limits (typically under 3 seconds).
- Database queries shall be optimized for efficient data retrieval.

Usability

- The system shall provide a simple and intuitive user interface.
- Users shall be able to navigate between modules easily.

Reliability

- The system shall ensure data consistency and integrity at all times.
- System operations shall handle errors gracefully without crashing.

Security

- Only authenticated users shall access the system.
- Role-based access control shall restrict unauthorized actions.
- Sensitive data such as passwords shall be securely stored.

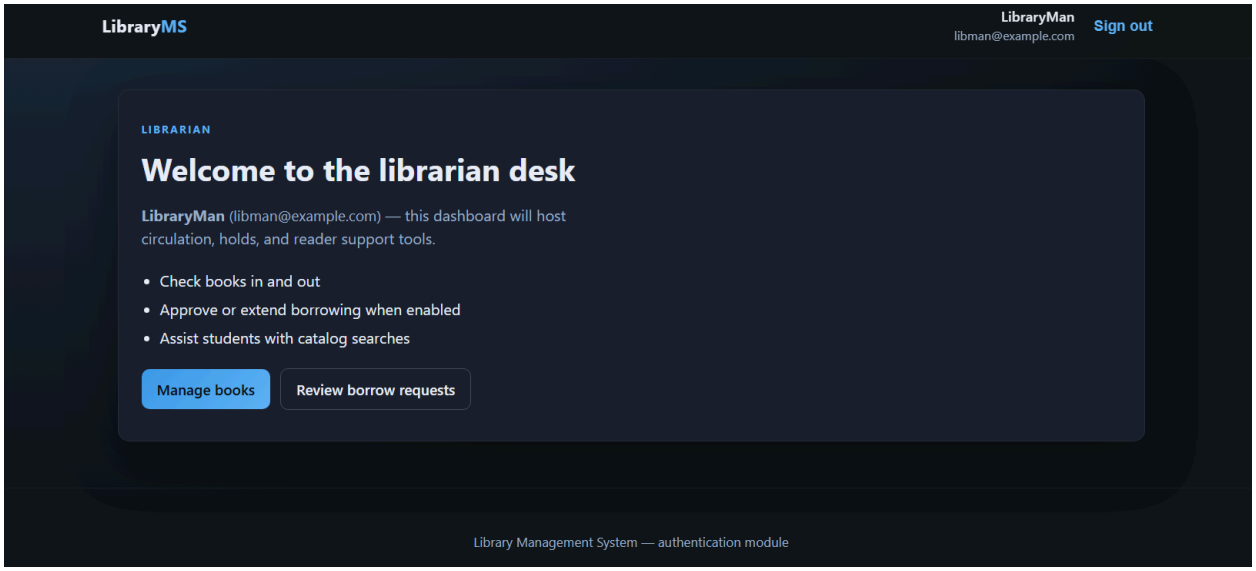
Scalability

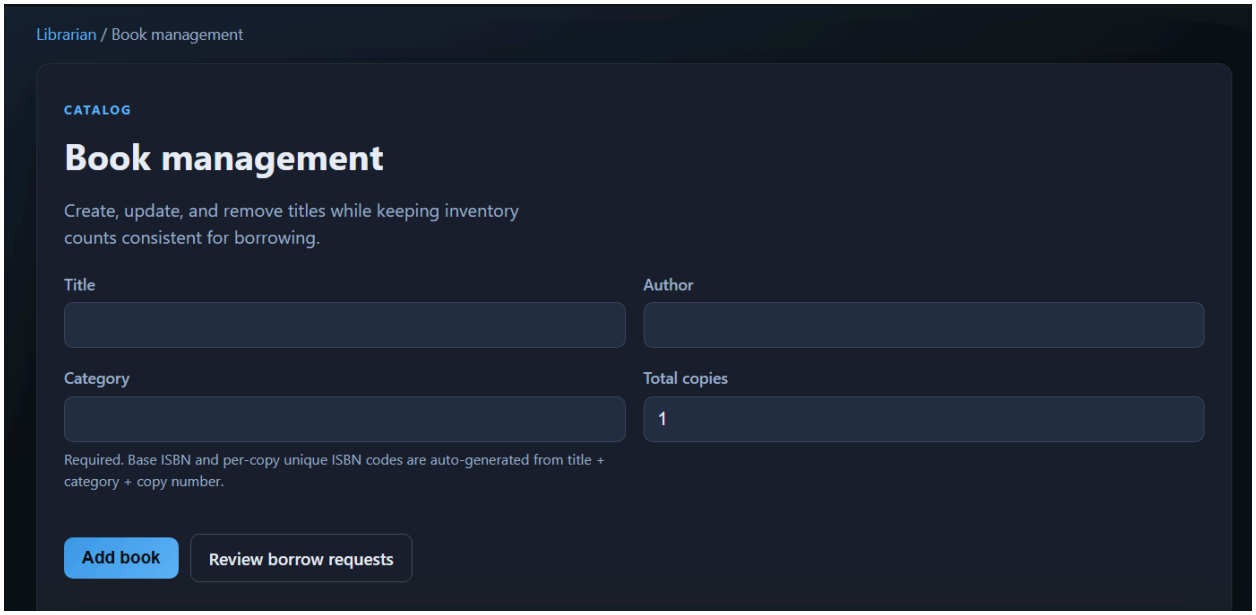
- The system shall be designed to handle increasing numbers of users and records.
- The database structure shall support future feature expansion.

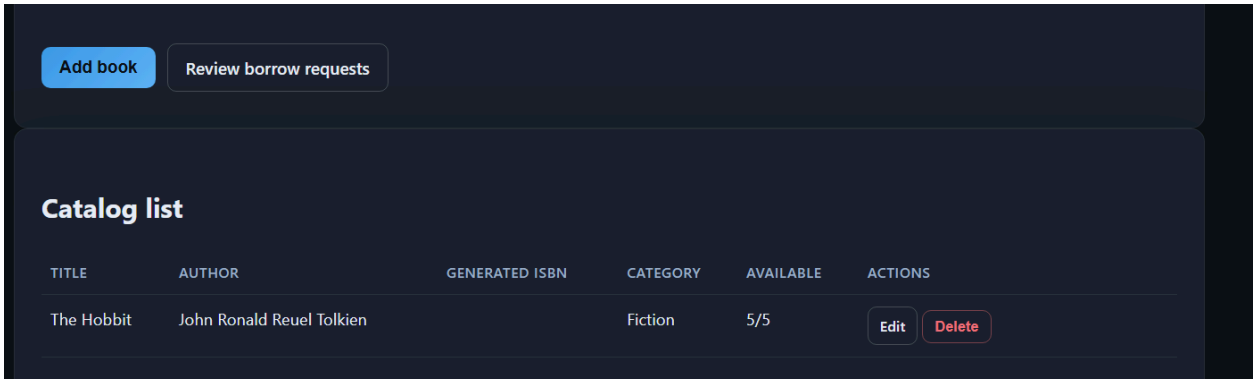
7. Iteration 2 Implementation Screenshots

The following screenshots demonstrate the implementation of Sprint 2 features:

Screenshot 1: Book Management Interface

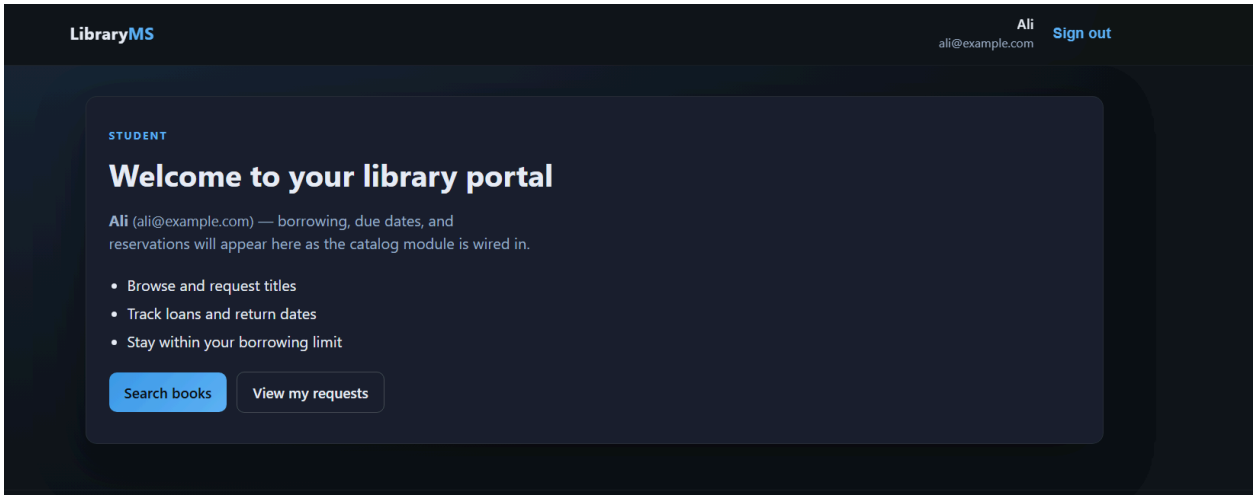


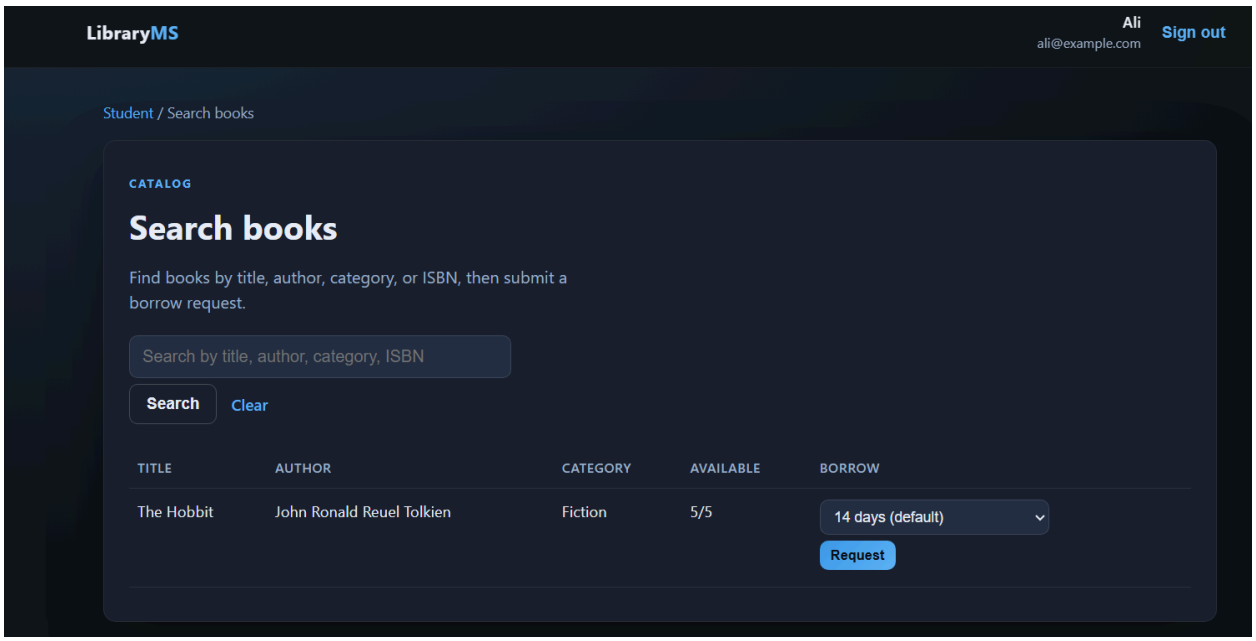




Caption: Interface for adding, updating, and deleting books by the librarian.

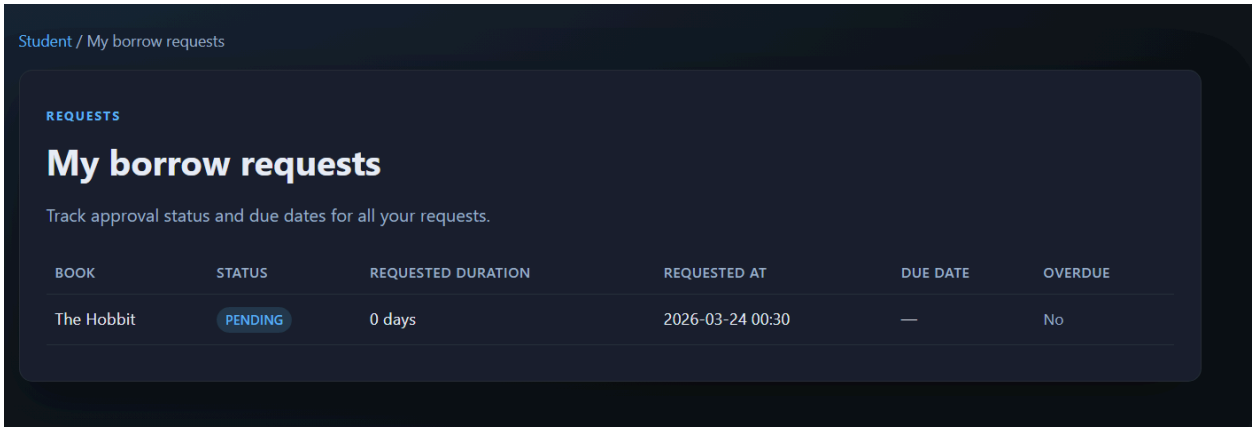
Screenshot 2: Book Search Page





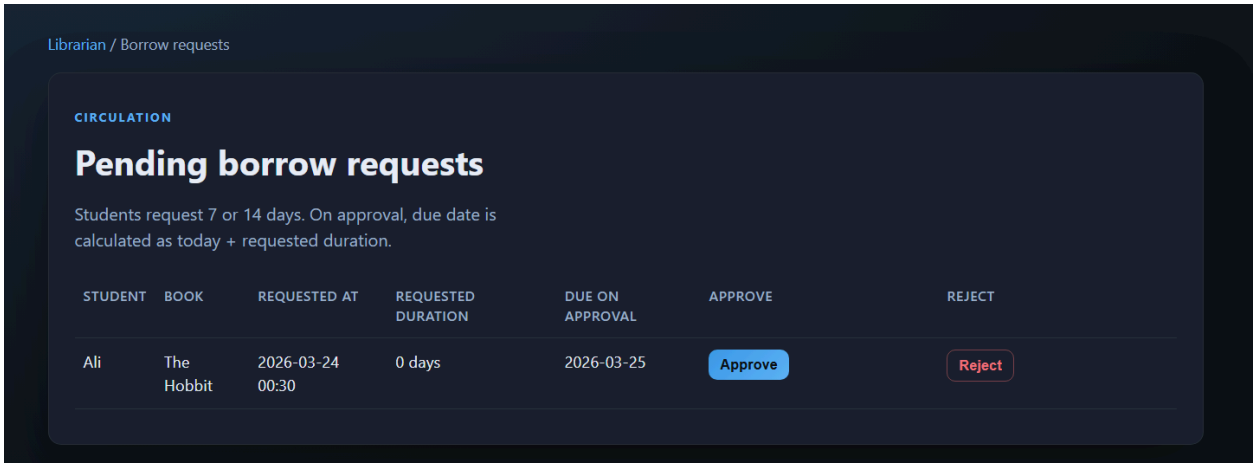
Caption: Student interface for searching books by title or author.

Screenshot 3: Borrow Request Submission



Caption: Student submitting a request to borrow a book.

Screenshot 4: Borrow Approval Dashboard



Caption: Librarian reviewing and approving/rejecting borrow requests.

8. Work Division for Sprint 2

Ali Bin Salman, 22i-0894

- Designed database schema for book and borrowing modules
- Implemented core backend APIs for book management
- Supervised system architecture and integration
- Managed project progress and coordination

Muhammad Rohaan, 22i-2327

- Implemented borrowing workflow logic

- Developed request approval and rejection system
- Handled backend service layer and integration
- Managed Scrum board and sprint tracking

Muhammad Atyab, 22i-1191

- Designed and implemented frontend UI for Sprint 2 features
- Developed book management and search interfaces
- Integrated frontend with backend APIs
- Performed system testing and debugging

Collaborative Work

All team members contributed to:

- Requirement analysis
- Sprint planning
- Debugging and integration
- Documentation preparation